

Azure Function

What is Azure Functions?

Azure Functions is a **serverless compute service** provided by Microsoft Azure that enables developers to execute small, event-driven pieces of code without managing infrastructure. It is designed to respond to various triggers, such as HTTP requests, timers, message queues, and changes in data sources.

As a **fully managed, scalable service**, Azure Functions automatically allocates computing resources as needed, making it ideal for lightweight, event-driven applications.

Key Benefits of Azure Functions

1. Serverless and Fully Managed

- Eliminates the need for infrastructure management, allowing developers to focus purely on writing code.
- Azure automatically handles scaling, monitoring, and security.

2. Cost-Efficient

- **Consumption Plan:** You only pay for the execution time of functions, reducing costs for infrequent workloads.
- **Premium and Dedicated Plans:** Suitable for high-performance and enterprise-level applications.

3. Event-Driven and Scalable

- Functions execute in response to various triggers (e.g., HTTP requests, database changes, event queues).
- Can scale **automatically** based on workload demand.

4. Multi-Language Support

- Supports **C#, JavaScript (Node.js), Python, Java, PowerShell, TypeScript, and F#**.
- Enables developers to choose their preferred language.

5. Rich Integration with Azure Services

- Works seamlessly with **Azure Blob Storage, Cosmos DB, Event Grid, Service Bus, SQL Database**, and more.
- Can be part of complex cloud-based workflows.

6. Supports Durable Functions

- Extends Azure Functions with **Durable Functions**, enabling **stateful workflows** and orchestrating multiple function executions.

7. Secure and Reliable

- Supports **Azure Active Directory (AAD) authentication**, Virtual Networks (VNETs), and managed identities.
- Fully monitored with **Azure Application Insights** for logging and performance tracking.

Use Cases of Azure Functions

1. API and Microservices

- Build **lightweight APIs** that handle HTTP requests without a full web server.
- Create **backend services** for mobile or web applications.

2. Background Task Processing

- Process **real-time data** asynchronously, such as image processing, file conversions, and data cleanup.
- Automate **cron jobs** with Azure Function **Timer Triggers**.

3. Event-Driven Automation

- **Trigger workflows** based on Azure Storage, Cosmos DB changes, or Service Bus messages.
- Automate tasks like **database updates**, notifications, or backups.

4. IoT and Real-Time Data Processing

- Process IoT data from **Azure IoT Hub** or **Event Hubs** in real-time.
- Analyze **sensor data**, detect anomalies, and trigger alerts.

5. Chatbots and AI Integration

- Integrate with **Azure Cognitive Services** for **chatbots**, **image recognition**, and **natural language processing (NLP)**.

6. CI/CD and DevOps Automation

- Automate **builds, deployments, and infrastructure provisioning** in a DevOps pipeline.
- Respond to repository changes with Azure DevOps or GitHub webhooks.

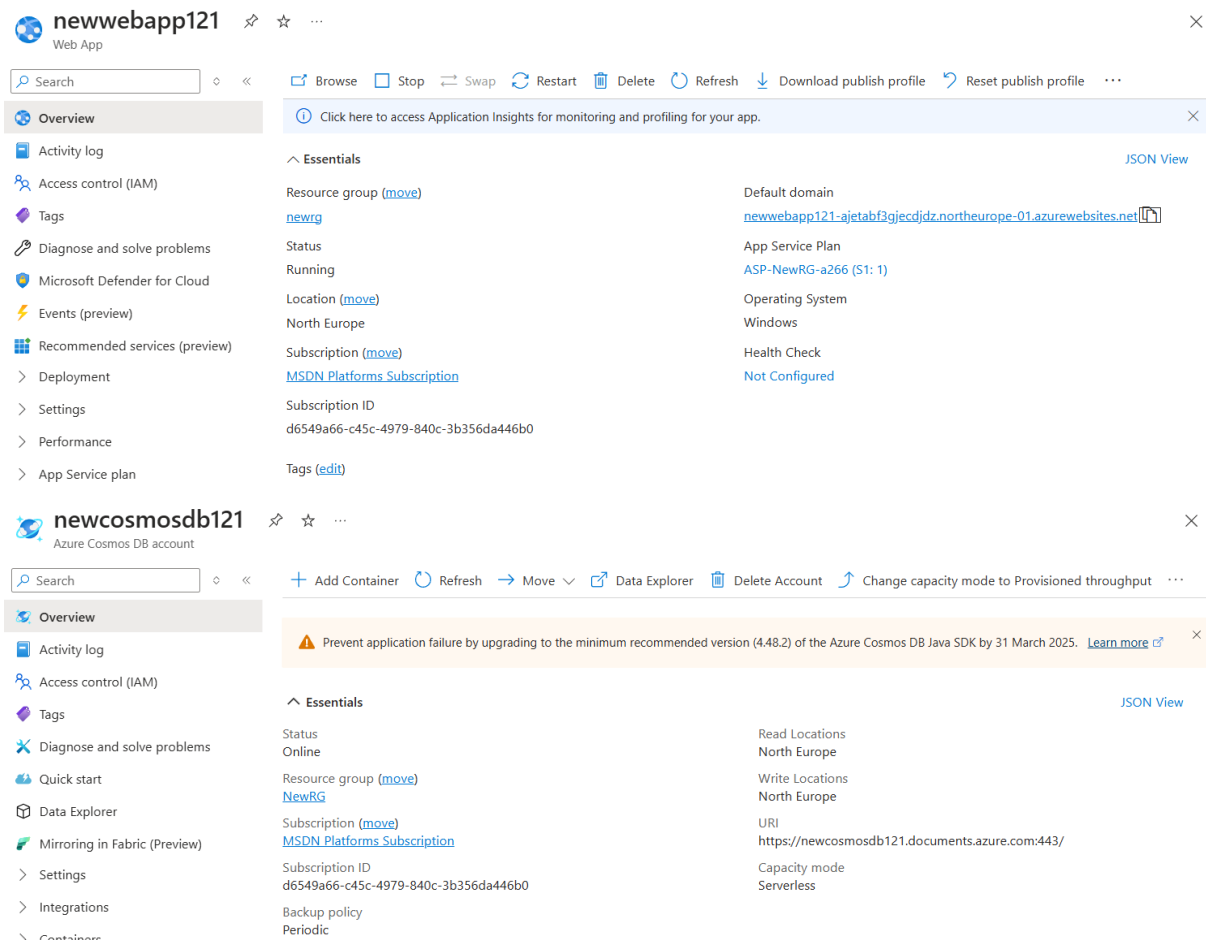
Conclusion

Azure Functions provides a **scalable, cost-effective, and event-driven** way to run cloud applications without managing servers. It is particularly useful for **API development, background processing, real-time automation, and serverless workflows**. With its **pay-per-use model** and **seamless integration with Azure services**, it is an excellent choice for modern cloud applications.

The process involves setting up an Azure Function App to process logs from an Azure Event Hub and store them in Azure Cosmos DB. First, create a Function App with an HTTP trigger, then configure a second function to listen to Event Hub events. Modify the connection strings in Visual Studio, update database details, and run the program locally. The function extracts relevant event data and writes it to Cosmos DB. Finally, publish the function to Azure, configure connection strings in the portal, and verify data ingestion. The end goal is an automated event processing pipeline for structured log storage.

😊 To begin with the Lab

1. To work with this lab, you need to have resources beforehand. This is the prerequisite for this lab.
2. You should have a web app, a cosmos DB account, and an event hub in place. All of the resources are covered in previous labs.
3. Then download the zip files attached with this lab.



The image displays two screenshots of the Azure portal. The top screenshot shows the 'Overview' page for a Web App named 'newwebapp121'. The page includes a search bar, a list of actions (Browse, Stop, Swap, Restart, Delete, Refresh, Download publish profile, Reset publish profile), and a sidebar with navigation links (Activity log, Access control (IAM), Tags, Diagnose and solve problems, Microsoft Defender for Cloud, Events (preview), Recommended services (preview), Deployment, Settings, Performance, App Service plan). The main content area is divided into 'Essentials' and 'JSON View' sections. The 'Essentials' section lists key information: Resource group (newrg), Status (Running), Location (North Europe), Subscription (MSDN Platforms Subscription), Subscription ID (d6549a66-c45c-4979-840c-3b356da446b0), and Tags (edit). The 'JSON View' section lists: Default domain (newwebapp121-ajetabf3gjecdjdznorth europe-01.azurewebsites.net), App Service Plan (ASP-NewRG-a266 (S1: 1)), Operating System (Windows), Health Check (Not Configured), and a link to 'Click here to access Application Insights for monitoring and profiling for your app.'

The bottom screenshot shows the 'Overview' page for an Azure Cosmos DB account named 'newcosmosdb121'. The page includes a search bar, a list of actions (Add Container, Refresh, Move, Data Explorer, Delete Account, Change capacity mode to Provisioned throughput), and a sidebar with navigation links (Activity log, Access control (IAM), Tags, Diagnose and solve problems, Quick start, Data Explorer, Mirroring in Fabric (Preview), Settings, Integrations, Containers). The main content area is divided into 'Essentials' and 'JSON View' sections. The 'Essentials' section lists key information: Status (Online), Resource group (NewRG), Subscription (MSDN Platforms Subscription), Subscription ID (d6549a66-c45c-4979-840c-3b356da446b0), Backup policy (Periodic), Read Locations (North Europe), Write Locations (North Europe), URI (https://newcosmosdb121.documents.azure.com:443/), and Capacity mode (Serverless). A warning message at the top states: 'Prevent application failure by upgrading to the minimum recommended version (4.48.2) of the Azure Cosmos DB Java SDK by 31 March 2025. Learn more'.

newevent121 Event Hubs Namespace

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Data Explorer
- Events
- Settings
 - Shared access policies
 - Scale
 - Geo-Recovery
 - Networking
 - Encryption
 - Identity

You can start generating test data or inspect data that has already been sent with the new Azure Event Hubs Data Explorer. Click on this message to try the feature!

Essentials

Resource group (move)	Created
NewRG	Wednesday, March 12, 2025 at 09:47:58 GMT+5:30
Status	Updated
Active	Wednesday, March 12, 2025 at 09:48:22 GMT+5:30
Location	Zone Redundancy
North Europe	Enabled
Subscription (move)	Pricing tier
MSDN Platforms Subscription	Standard
Subscription ID	Throughput Units
d6549a66-c45c-4979-840c-3b356da446b0	1 unit
Host name	Auto-inflate throughput units
newevent121.servicebus.windows.net	Disabled
Tags (edit)	Local Authentication
Add tags	Enabled

[JSON View](#)

4. Now we need to create a function app. Click on Create.

Function App

Microsoft



Function App

Microsoft | Azure Service

★ 4.0 (1504 ratings)

Plan

Function App

Create

5. Then you have to choose App service as your hosting option. Click on Select.

Create Function App ...



Select a hosting option

These options determine how your app scales, resources available per instance, and pricing. [Learn more about Functions hosting options](#)

Hosting plans	☐ Flex Consumption	☐ Consumption	☐ Functions Premium	☒ App Service	☐ Container Apps environment
	Get high scalability with compute choices, virtual networking, and pay-as-you-go billing.	Pay for compute resources when your functions are running (pay-as-you-go).	Deploy multiple function apps on the same plan with event-driven scaling.	Run web apps and function apps on the same plan with more compute choices and pay for the instances of the plan.	Host function apps with other containerized microservices and pay for compute capacity.
Scale to zero	✓	✓	-	-	✓
Scale behavior	Fast event-driven	Event-driven	Event-driven	Metrics based	Event-driven with KEDA
Virtual networking	✓	-	✓	✓	✓
Dedicated compute and prevent cold start	Optional with Always Ready	-	Minimum of 1 instance required	Minimum of 1 instance required	Optional with minimum replicas
Max scale out (instances)	1000	200	100	30	300

Select

6. Now give it a name to your function and choose your resource group.

Basics Storage Networking Monitoring Deployment Tags Review + create

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ

MSDN Platforms Subscription

Resource Group * ⓘ

NewRG

[Create new](#)

Instance Details

Function App name *

newFunctionapp121

.azurewebsites.net



Try a secure unique default hostname (preview). [More about this update](#)

Do you want to deploy code or container image? *



Code



Container Image

7. Choose the runtime stack as dot Net and choose your region.

Runtime stack *

Version *

Region *

 Not finding your App Service Plan? Try a different region or select your App Service Environment.

Operating System * ☐ Linux ☒ Windows

Environment details

Windows Plan (North Europe) * ⓘ

[Create new](#)

Pricing plan **Standard S1** (100 total ACU, 1.75 GB memory, 1 vCPU)

8. It will also create a storage account for you. Then in the monitoring section turn on the application insights feature.

Basics Storage Networking Monitoring Deployment Tags Review + create

Storage

When creating a function app, you must create or link to a general-purpose Azure Storage account that supports Blobs, Queue, and Table storage. [Learn more](#) 

Storage account *

[Create new](#)

Basics Storage Monitoring Deployment Tags Review + create

Application Insights

Azure Monitor application insights is an Application Performance Management (APM) service for developers and DevOps professionals. Enable it below to automatically monitor your application. It will detect performance anomalies, and includes powerful analytics tools to help you diagnose issues and to understand what users actually do with your app. Your bill is based on amount of data used by Application Insights and your data retention settings. [Learn more](#) 

[App Insights pricing](#) 

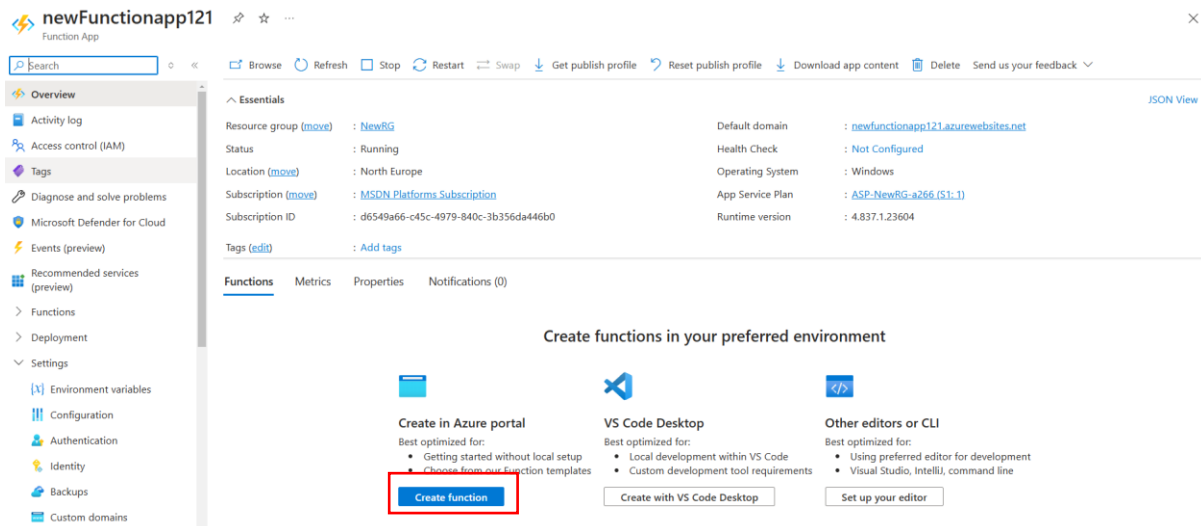
Enable Application Insights * ☐ No ☒ Yes

Application Insights *

[Create new](#)

Region

9. In your function app you can create a function. Click on Create.



10. Here we have multiple functions to choose from, for now, to keep things simple we will choose HTTP trigger. Click on next.

Create function

1 Select a template 2 Template details

Use a template to create a function. Triggers describe the type of events that invoke your functions. [Learn more](#)

Name	Trigger
HTTP trigger	A function that will be run whenever it receives an HTTP request, responding based on data in the body or query string
Timer trigger	A function that will be run on a specified schedule
Azure Queue Storage trigger	A function that will be run whenever a message is added to a specified Azure Storage queue
Azure Service Bus Queue trigger	A function that will be run whenever a message is added to a specified Service Bus queue
Azure Service Bus Topic trigger	A function that will be run whenever a message is added to the specified Service Bus topic
Azure Blob Storage trigger	A function that will be run whenever a blob is added to a specified container
Azure Event Hub trigger	A function that will be run whenever an event hub receives a new event
Azure Cosmos DB trigger	A function that will be run whenever documents change in a document collection
IoT Hub (Event Hub)	A function that will be run whenever an IoT Hub receives a new event from IoT Hub (Event Hub)

11. Now give a name to your function and then click on create.

Create function

- 1 Select a template
- 2 Template details**

We need more information to create the function. [Learn more](#)

Function template

HTTP trigger

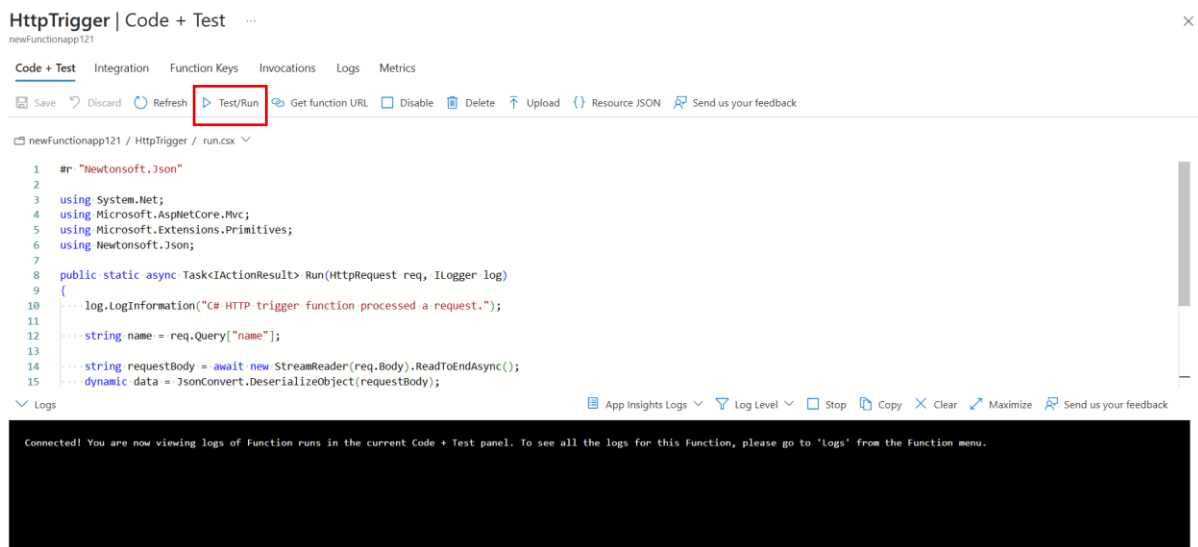
Function name *

HttpTrigger

Authorization level * ⓘ

Function

12. Once your function is created you will be directly inside it and you will get a default C# code. Click on Test/Run.



13. Then in the Input we need to use Get as our method then in the query parameters we can give a name and value.

Input Output

Provide parameters to test the HTTP request. Results can be found in the Output tab.

HTTP method * ⓘ	GET
Key *	_master (Host key)

Query parameters

Name	Value	
name	Bob	
		

14. And in the output, you can get the status of 200 OK.

Test/Run



Input Output

HTTP response code	200 OK
HTTP response content	
"This HTTP triggered function executed successfully. Pass a name in the query string or in the request body for a personalized response."	

15. So, below is the structure diagram of what we will doing next. First, if you remember from the previous lab we are sending logs from the Azure web app to the Azure Event hubs.
16. Now we want the Azure function should consume the event from the event hub and then send it onto a container in Azure Cosmos DB.



Azure Web App



Azure Event Hub



Azure Function



Azure Cosmos DB

Diagnostic setting

17. We also looked at the triggers that are available in Azure Function. Previously we created a trigger based on HTTP now we can create a trigger based on the Azure event hub.
18. But this time we have a program, so this is an Azure function program defined in Visual Studio that can go ahead and read the events in Azure Event hubs.
19. So, you will find two programs attached to this lab named **Part 1 and Part 2**. First, extract this **part 1 program** and open it in **Visual Studio 2022**.
20. In the program on line number 16 you need to change the name of the event hub with your event hub's name.
21. Then you need to move to the local settings JSON file to change the connection string.

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using Azure.Messaging.EventHubs;
7  using Microsoft.Azure.WebJobs;
8  using Microsoft.Extensions.Logging;
9  using Newtonsoft.Json.Linq;
10
11  namespace ProcessEventHub
12  {
13      0 references
14      public static class ProcessMessage
15      {
16          [FunctionName("ProcessMessage")]
17          public static async Task Run([EventHubTrigger("loghub", Connection = "hubconnection")] E
18          {
19              var exceptions = new List<Exception>();
20
21              // Replace these two lines with your processing logic.
22              foreach (EventData eventData in events)
23              {

```

22. From the event hub go to the shared access policy and from your listen policy copy the connection string.

SAS Policy: ListenPolicy

Event Hub



Regenerate primary key Regenerate secondary key Delete

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims

☐ Manage

☐ Send

☒ Listen

Primary key

.....

Secondary key

.....

Primary connection string

Endpoint=sb://newevent121.servicebus.windows.net/;SharedAccessKeyName=L...

Secondary connection string

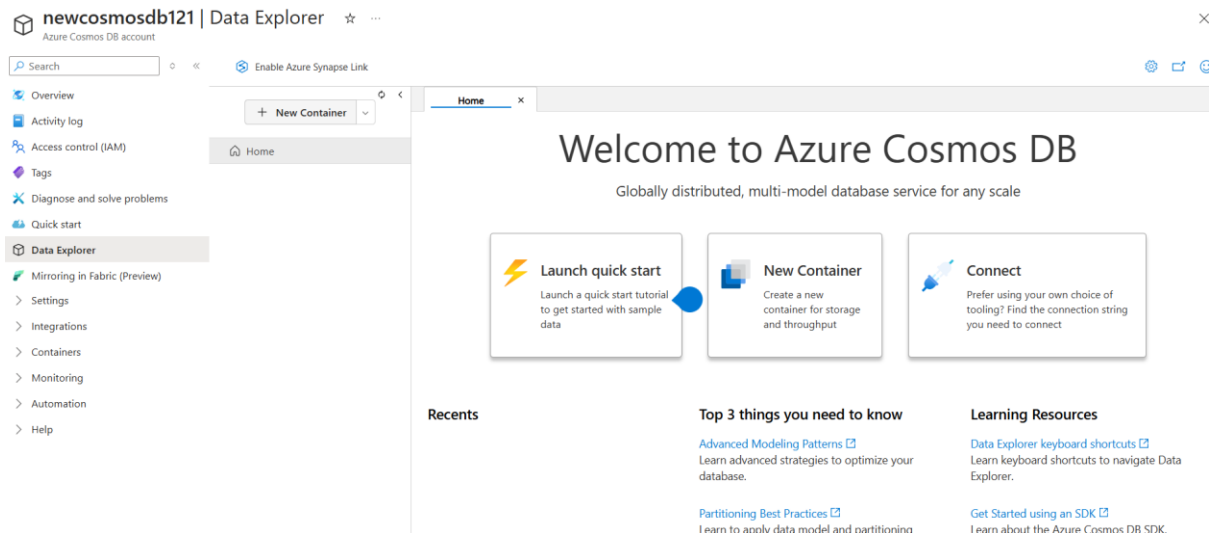
.....

SAS policy ARM ID

/subscriptions/d6549a66-c45c-4979-840c-3b356da446b0/resourceGroups/NewRG/...

23. Then in the program paste it on line number 6 in the local.setting.json file. Also, remove the entity path from the connection string because we are already using the name.

```
local.settings.json* X ProcessMessage.cs
Schema: https://json.schemastore.org/local.settings.json
1  |
2  |
3  |
4  |
5  |
6  | :cessKey=Mt6br2QB9ZFS16I0JJbpcXYg6I3cGKB4u+AEhA8s158=;EntityPath=loghub"
7  |
8  |
```

27. Then give a Database ID and Container ID. For the Partition key say CIP.

New Container ✕

*** Database id** ⓘ

☒ Create new ☐ Use existing

logdb

*** Container id** ⓘ

weblogs

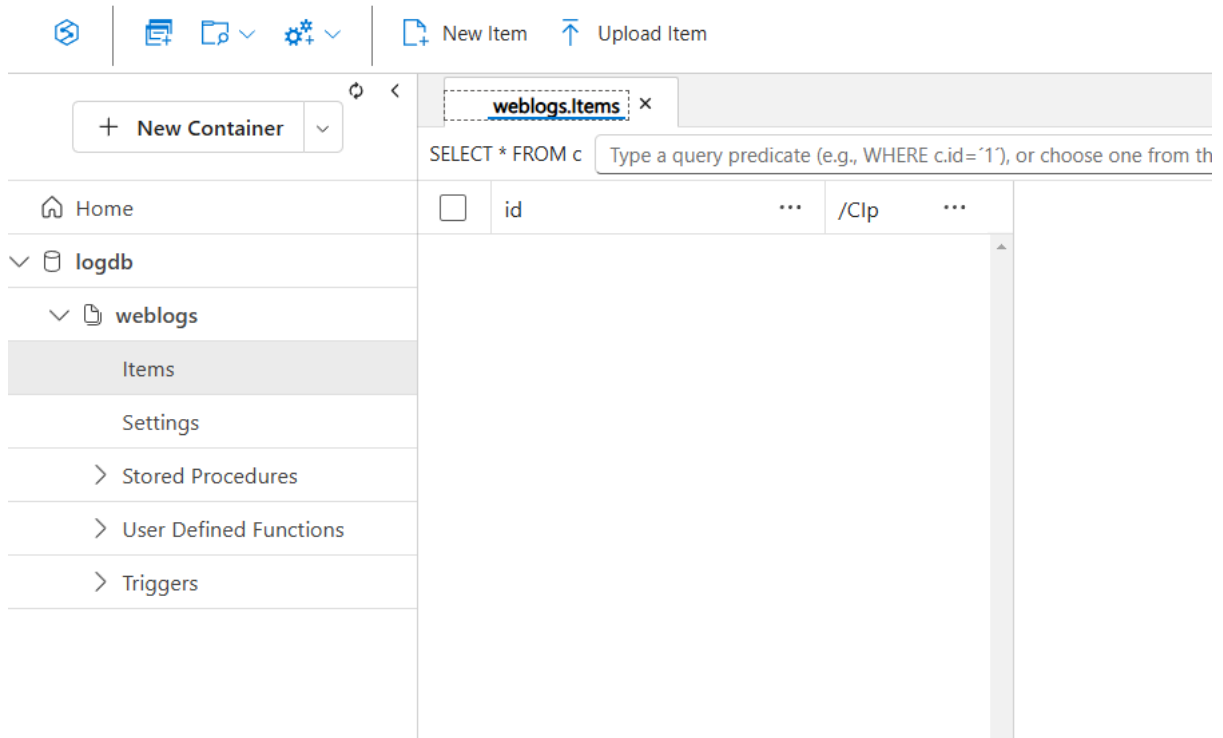
*** Partition key** ⓘ

/CIP

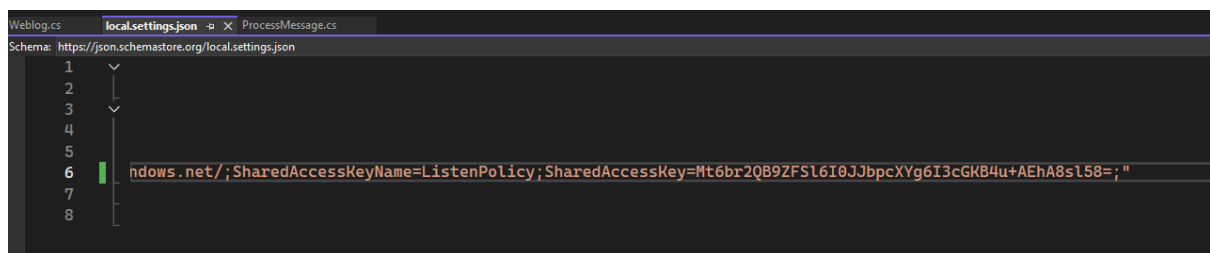
Add hierarchical partition key

Unique keys ⓘ

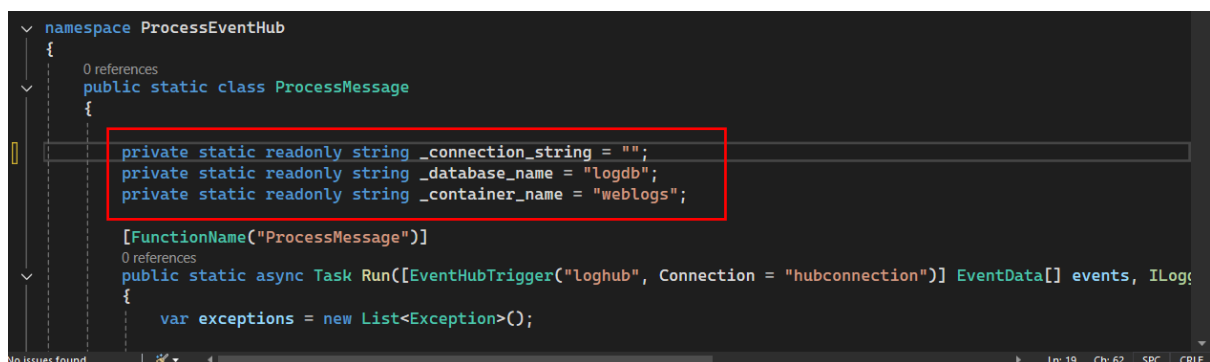
28. You will not have any items within the container, our Azure Function will actually go ahead and pick up the events from the Event Hub and then only take the data that is required and send it as individual items in the container in the logdb database.



29. In the second program we also need to change the connection string and remove the entity path in the local.settings.json file.

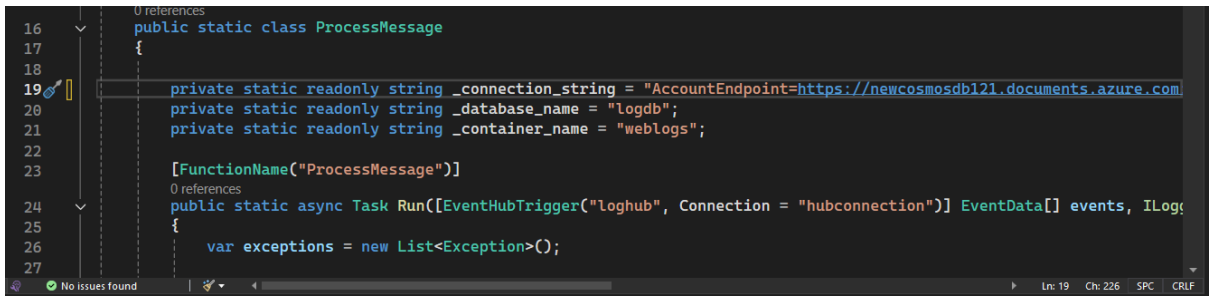


30. Then we need to change the database name container name and give the connection string for the Cosmos DB.



31. In your cosmos DB account go to keys and copy the connection string from there.

32. Then just save everything and run the program.



```
Functions:
```

```
    ProcessMessage: eventHubTrigger
```

```
For detailed output, run func with --verbose flag.
```

```
[2025-03-12T06:42:48.278Z] Host lock Lease acquired'
```

```
[2025-03-12T06:44:10.489Z] Executing 'ProcessMessage' (Reason='null'), Id=b7f6ff29-42c1-4bfd-a3e1-65bfc8e1b27c)
```

```
[2025-03-12T06:44:10.411Z] Trigger Details: PartitionId: 0, Offset: 21536-21536, EnqueueTimeUtc: 2025-03-12T06:44:10.165000+00:00-2025-03-12T06:44:10.165000+00:00, SequenceNumber: 12-12, Count: 1
```

```
Item created
```

```
Item created
```

```
[2025-03-12T06:44:14.971Z] Executed 'ProcessMessage' (Succeeded, Id=b7f6ff29-42c1-4bfd-a3e1-65bfc8e1b27c, Duration=4594ms)
```

```
[2025-03-12T06:46:11.890Z] Executing 'ProcessMessage' (Reason='null'), Id=13dc17f3-ba0b-4f2b-8898-3d8a5093d8bb)
```

```
[2025-03-12T06:46:11.890Z] Trigger Details: PartitionId: 1, Offset: 22624-22628, EnqueueTimeUtc: 2025-03-12T06:46:11.718000+00:00-2025-03-12T06:46:11.718000+00:00, SequenceNumber: 7-7, Count: 1
```

```
Item created
```

```
Item created
```

```
Item created
```

```
Item created
```

```
Item created
```

```
Item created
```

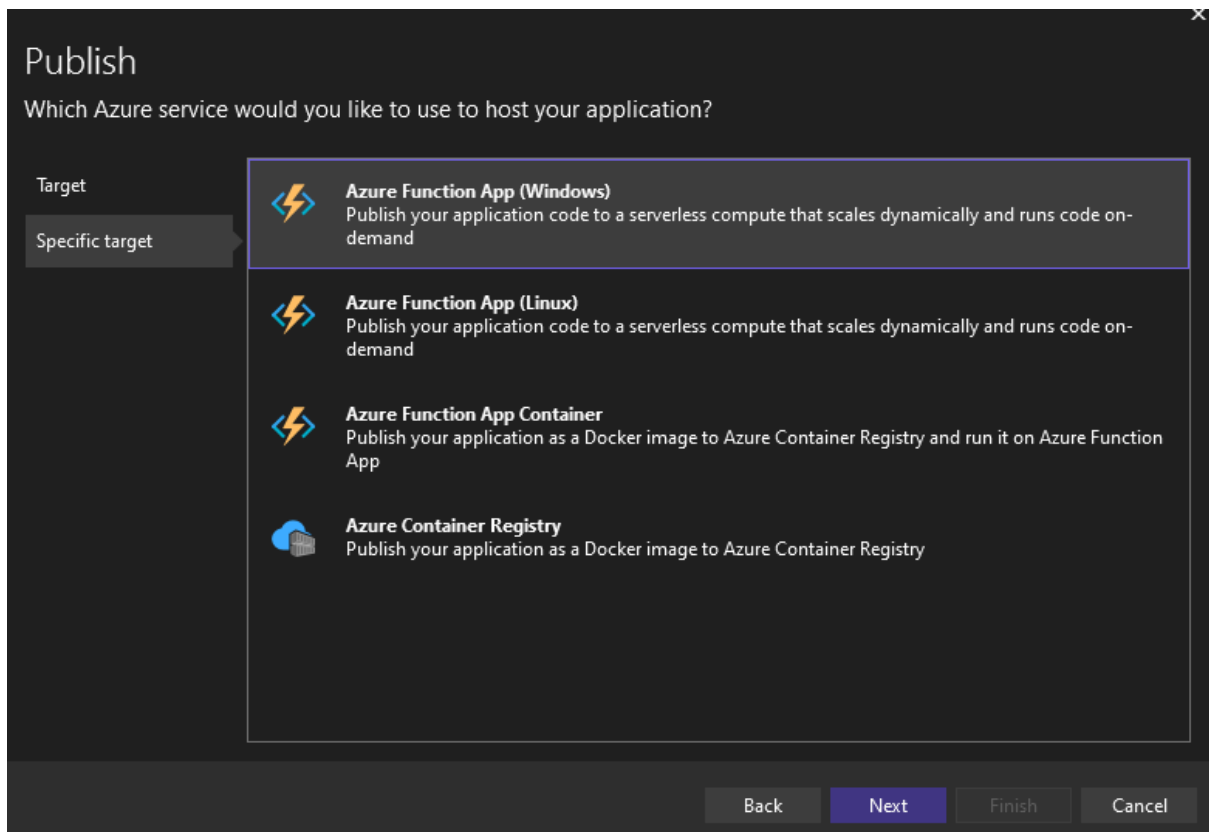
```
Item created
```

```
Item created
```

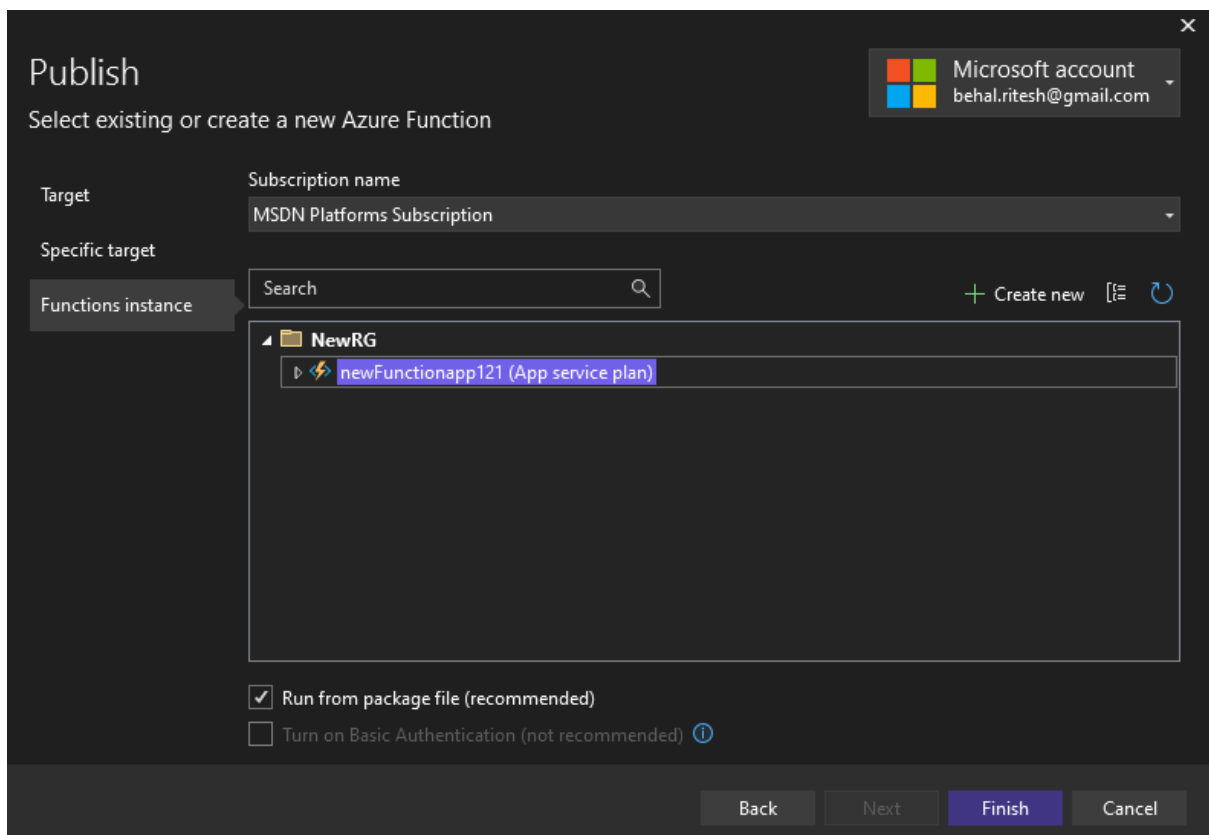
```
Item created
```

```
[2025-03-12T06:46:17.839Z] Executed 'ProcessMessage' (Succeeded, Id=13dc17f3-ba0b-4f2b-8898-3d8a5093d8bb, Duration=5149ms)
```

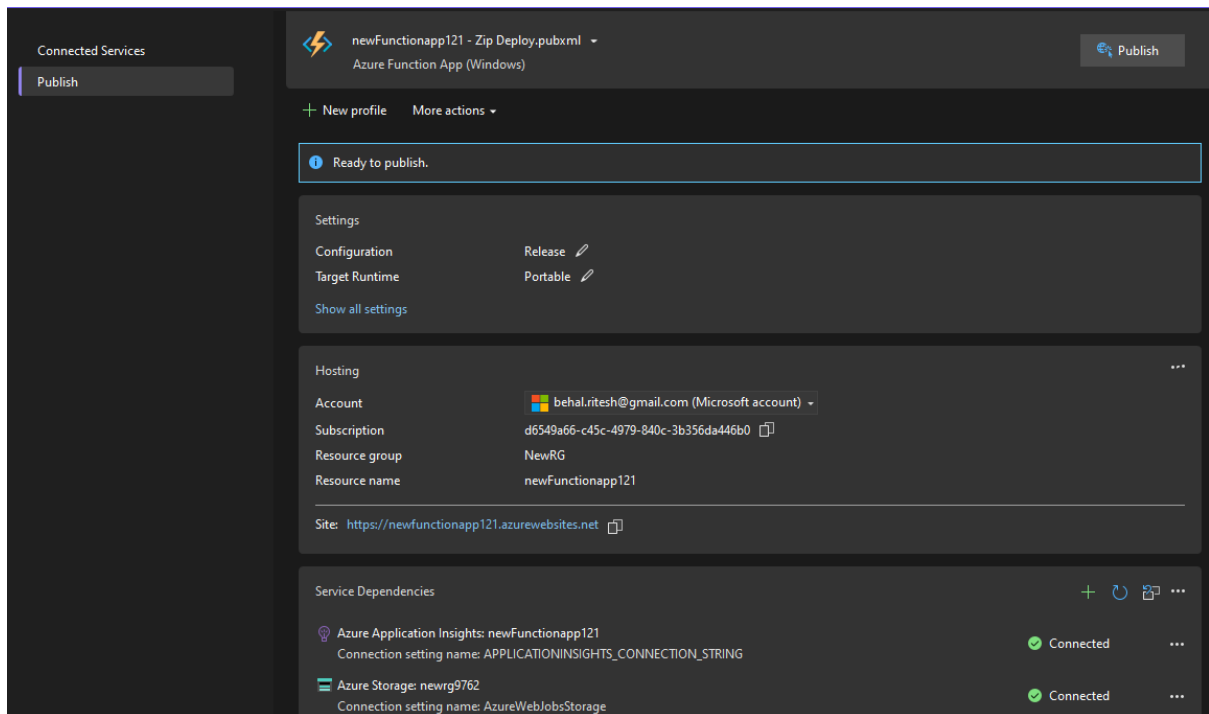
34. You can see that the items have been added. Currently, we have 15 items.



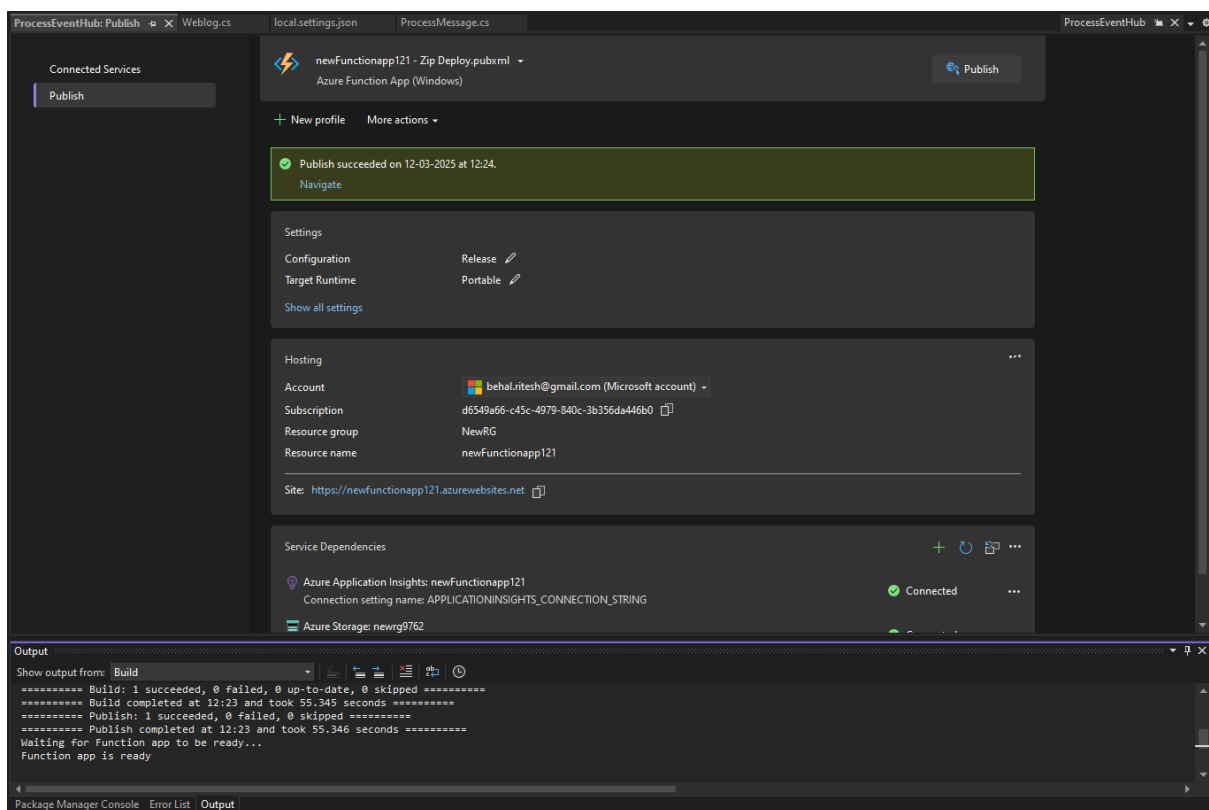
38. Choose your function from the resource group and click on finish.



39. In the end click on publish and wait for the function to get ready.



40. Here you can see that our function and we can view this in the Azure portal.



41. Our function is ready and enabled but we are getting an error that the hub connection does not exist. So, to remove this error we need to go to the environment variables tab in the function app. Click on add connection strings.

newFunctionapp121 Function App

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Microsoft Defender for Cloud
- Events (preview)
- Recommended services (preview)
- Functions
- Deployment
- Settings
 - Environment variables
 - Configuration
 - Authentication
 - Identity
 - Backups
 - Custom domains
 - Certificates

Microsoft.Azure.WebJobs.Extensions.EventHubs: EventHub account connection string 'hubconnection' does not exist. Make sure that it is a defined App Setting.

[NewRG](#)

Status: Running

Location: North Europe

Subscription: MSDN Platforms Subscription

Subscription ID: d6549a66-c45c-4979-840c-3b356da446b0

Tags: Add tags

[newfunctionapp121.azurewebsites.net](#)

Health Check: Not Configured

Operating System: Windows

App Service Plan: ASP-NewRG-a266 (S1:1)

Runtime version: Error

Functions Metrics Properties Notifications (1)

Set up local environment Refresh

Filter by name...

Name	Trigger	Status	Monitor
ProcessMessage	Event Hub	Enabled	Invocations and more

newFunctionapp121 | Environment variables Function App

Search

Settings

- Environment variables
- Configuration
- Authentication
- Identity
- Backups
- Custom domains
- Certificates

App settings Connection strings

Search Add Refresh Show values Advanced edit Pull reference values

Name	Value	Deployment slot setting	Type	Source
------	-------	-------------------------	------	--------

42. Now to get the connection strings we can either go to the event and copy the connection string from there or we can open our program and from the local.settings.json file, we can get the hub connection.

```

ProcessEventHub: Publish Weblog.cs local.settings.json ProcessMessage.cs ProcessEventHub
Schema: https://json.schemastore.org/local.settings.json
1  {
2    "IsEncrypted": false,
3    "Values": {
4      "AzureWebJobsStorage": "UseDevelopmentStorage=true",
5      "FUNCTIONS_WORKER_RUNTIME": "dotnet",
6      "hubconnection": "Endpoint=sb://newevent121.servicebus.windows.net/;SharedAccessKeyName=ListenPolicy;SharedAccessKey=Hl
7    }
8  }

```

43. Now you need to give a name and, in the value, give the connection string then choose custom for the type. Click on apply.

Add/Edit connection string

Name *

Value

Type *

At runtime, connection strings are available as environment variables. [Learn more](#)

Deployment slot setting ☐

44. What we can do now is, refresh our web app so that logs get generated and wait for some time.

45. As you can see below the error has been gone now

The screenshot shows the Azure portal interface for a new Function App. The left sidebar contains navigation options like Overview, Activity log, Access control, Tags, Diagnose and solve problems, Microsoft Defender for Cloud, Events, Recommended services, Functions, Deployment, and Settings. The main area displays the 'Essentials' tab for the Function App, showing details such as Resource group (NewRG), Status (Running), Location (North Europe), Subscription (MSDN Platforms Subscription), Subscription ID (d6549a66-c45c-4979-840c-3b356da446b0), Default domain (newfunctionapp121.azurewebsites.net), Health Check (Not Configured), Operating System (Windows), App Service Plan (ASP-NewRG-a266 (\$1:1)), and Runtime version (4.837.1.23604). Below this, the 'Functions' tab is active, showing a table with columns: Name, Trigger, Status, and Monitor. The table contains one entry: 'ProcessMessage' with an 'Event Hub' trigger, 'Enabled' status, and a 'Monitor' link.

46. When we return to the items in the cosmos DB, we can see multiple items added here.

The screenshot shows the Azure portal interface for a Cosmos DB database named 'logdb'. The left sidebar shows the navigation tree with 'logdb' expanded, showing 'weblogs' and 'Items'. The main area displays the 'Items' view for the 'weblogs' collection. A table shows a list of documents with columns: id, /C/Ip, and a status column. The table contains 10 rows of data, each with a unique ID and a corresponding IP address. The status column shows '172.17.32.9', '127.0.0.1', and '103.226.201...' for the first three rows. The table is paginated, showing 'Load more' at the bottom. The top of the interface shows the 'SELECT * FROM c' query and a search bar.